

Object-oriented Programming

Week 6

Storage Classes

- To fully define a variable one needs to mention not only its ‘type’ but also its ‘storage class’.
- In other words, not only do all variables have a data type, they also have a ‘storage class’.
- storage classes have defaults, If we don’t specify the storage class of a variable in its declaration,
- There are basically two kinds of locations in a computer where such a value may be kept—
- Memory and CPU registers.
- It is the variable’s storage class that determines in which of these two locations the value is stored.

Storage Classes

- Moreover, a variable's storage class tells us:
- (a) Where the variable would be stored.
- (b) What will be the initial value of the variable, if initial value is not specifically assigned.(i.e. the default initial value).
- (c) What is the scope of the variable; i.e. in which functions the value of the variable would be available.
- (d) What is the life of the variable; i.e. how long would the variable exist.

Storage-class Specifiers

1. Auto
2. Register
3. Extern
4. Static
5. Mutable

static storage class

- Storage – Memory.
- Default initial value – Zero.
- Scope – Local to the block in which the variable is defined.
- Life – Value of the variable persists between different function calls.

A simple function call

```
void staticDemo()
```

```
{
```

```
    int val = 0;
```

```
    ++val;
```

```
    cout << "val = " << val << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    staticDemo();           // prints val = 1
```

```
    staticDemo();           // prints val = 1
```

```
    staticDemo();           // prints val = 1
```

```
}
```

Static Local Variables

```
void staticDemo()
{
    static int val = 0;
    ++val;
    cout << "val = " << val << endl;
}

int main()
{
    staticDemo();           // prints val = 1
    staticDemo();           // prints val = 2
    staticDemo();           // prints val = 3
}
```

- `static void counter() {`
- `static int count = 0; // Stored in Data Segment, persists`
- `++count;`
- `cout << "Count: " << count << endl;`
- `}`

Memory Layout

Memory Segment	Purpose	Examples
Text (Code) Segment	Stores machine code (compiled instructions).	Functions (including <code>main()</code> , and <code>static</code> member functions).
Data Segment (Global & Static)	Stores global/static variables.	<code>static int x;</code> (initialized and uninitialized data).
Heap Segment	Stores dynamically allocated memory (<code>new</code> / <code>malloc</code>).	<code>int* p = new int[10];</code>
Stack Segment	Stores function calls, local variables, return addresses.	Local variables inside functions.

Static main

main() is not static because:

- It must be accessible globally as the program's entry point.
- The C++ standard requires it to have external linkage.
- It is not a member of a class, so static has no meaning here.

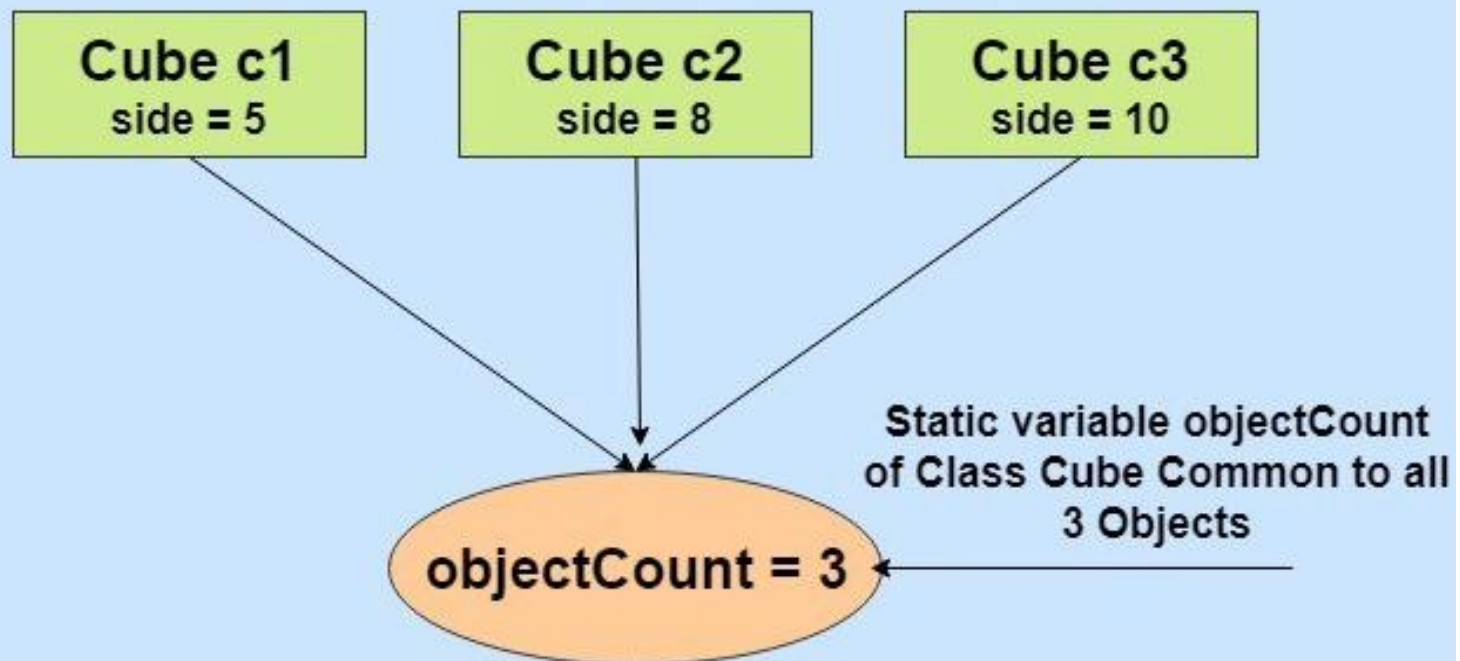
Static Data Member

- A data member of a class can be qualified as static
- It **exists even if no objects are created**, but it is **initialized only once**.
- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is visible only within the class, but its lifetime is the entire program

Static Data Member

- A static variable is normally used to maintain value common to the entire class. For e.g, to hold the count of objects created. Note that the type and scope of each static member variable must be declared outside the class definition. This is necessary because the static data members are stored separately rather than as a part of an object.

3 Objects of Cube Class with their individual instance variable



Declaration

```
static data_type member_name;
```

Defining the static data member

It should be defined outside of the class following this syntax:

- **data_type class_name :: member_name =value;**

```
#include <iostream>
using namespace std;
class Demo{
    public:
        static int ABC;};

//defining
int Demo :: ABC ;
int main()
{
    cout<<"\nValue of ABC: "<<Demo::ABC;
    return 0;
}
```

- `#include <iostream>`
 - `using namespace std;`
 - `class Demo{`
 - `public:`
 - `static int ABC;;`
 - `//defining`
 - `int Demo :: ABC =10;`
- `int main()`
- `{`
 - `cout<<"\nValue of ABC: "<<Demo::ABC;`
 - `return 0;`
 - `}`

Static Member Function

A static function can have access to only other static members (function or variable) declared in the same class.

- A static member function can be called using the
- class name (instead of its object) as follows-

Class_name::Function_name();

```
#include <iostream>
using namespace std;
class Demo{
private:
static int X;
public:
static void fun(){
cout <<"Value of X: " << X << endl;
}
};//defining
int Demo :: X =10;
int main(){
Demo X; X.fun();
return 0;
}
```

```
#include <iostream>
using namespace std;
class Demo{
private:
int a;
static int X;
public:
static void fun(){
cout <<"Value of X: " << X << endl;
cout <<"Value of a: " << a << endl; //[Error] invalid use of member
'Demo::a' in static member function
}
};//defining
int Demo :: X =10;
int main(){
Demo X; X.fun();
return 0;
}
```

Non static member func

```
#include <iostream>
using namespace std;
class Demo{
private:
int a;
static int X;
public:
void fun(){
cout <<"Value of X: " << X << endl;
cout <<"Value of a: " << a << endl;}
};//defining
int Demo :: X =10;
int main(){
Demo X; X.fun();
return 0;
}
```

C:\Users\Nida Munawar\Documents\Untitled1.exe

Value of X: 10

Value of a: 0

Process exited after 0.2311 seconds with return value 0

Press any key to continue . . .

Difference between a static variable and global variable

- All global variables are static, but all static variables are not necessarily global.
- static variables have visibility limited to the function in which declared.
- What all static variables (including global) have in common is that they have a lifetime that lasts the duration of the program. They are initialized just once, when the program begins.

Non static object

```
#include <iostream>
using namespace std;
class Counter {
public:
    Counter() {
        cout << "Constructor Called" << endl;
        count = 0; }
    ~Counter() {
        cout << "Destructor Called" << endl;
    }
    void increment() {
        count++;
        cout << "Count: " << count << endl;
    }
private:
    int count;
};

void testFunction() {
    Counter obj;
    obj.increment();
}
```

```
}
```

```
int main() {  
    cout << "Inside Main" << endl;  
  
    testFunction(); // First call  
    testFunction(); // Second call  
    testFunction(); // Third call  
  
    cout << "Exiting Main" << endl;  
    return 0;  
}
```

D:\fall 2024\OneDrive\Docum ×

+

▼

```
Inside Main
Constructor Called
Count: 1
Destructor Called
Constructor Called
Count: 1
Destructor Called
Constructor Called
Count: 1
Destructor Called
Exiting Main
```

Static object

```
#include <iostream>
using namespace std;
class Counter {
public:
    Counter() {
        cout << "Constructor Called" << endl;
        count = 0; }

    ~Counter() {
        cout << "Destructor Called" << endl;
    }

    void increment() {
        count++;
        cout << "Count: " << count << endl;
    }

private:
    int count;
};

void testFunction() {
    static Counter obj; // Static object inside function
    obj.increment();    // Retains value across function calls
}
```

```
int main() {  
    cout << "Inside Main" << endl;  
  
    testFunction(); // First call  
    testFunction(); // Second call  
    testFunction(); // Third call  
  
    cout << "Exiting Main" << endl;  
    return 0;  
}
```



D:\fall 2024\OneDrive\Docum X



Inside Main

Constructor Called

Count: 1

Count: 2

Count: 3

Exiting Main

Destructor Called

Why Can't Constructors Be Static?

A constructor is responsible for initializing an instance of a class.

- Static functions belong to the class, not to any instance.
- A constructor must always be tied to an object being created,
- which contradicts the purpose of a static function.
- If a constructor were static, it wouldn't be able to initialize non-static members.

Automatic Storage Class

- The **auto** storage class is the default storage class for all local variables.
- Storage – Memory.
- Default initial value – An unpredictable value, which is often called a garbage value.
- Scope – Local to the block in which the variable is defined.
- Life – Till the control remains within the block in which the variable is defined.

Automatic Storage Class

- {
- int count;
- OR
- auto int count;
- }

Register Storage Class

- The features of a variable defined to be of **register** storage class are as under:
- Storage – CPU registers.
- Default initial value – Garbage value.
- Scope – Local to the block in which the variable is defined.
- Life – Till the control remains within the block in which the variable is defined.

- A value stored in a CPU register can always be accessed faster than the one that is stored in memory.
- {
- Register int count;
- }

External Storage Class

- The **extern** storage class is used to give a reference of a global variable that is visible to ALL the program files. When you use 'extern' the variable cannot be initialized as all it does is point the variable name at a storage location that has been previously defined.
- When you have multiple files and you define a global variable or function, which will be used in other files also, then *extern* will be used in another file to give reference of defined variable or function. Just for understanding *extern* is used to declare a global variable or function in another file.

main.cpp

- `#include <iostream>`
- `int count ;`
- `extern void write_extern();`
-
- `main() {`
- `count = 5;`
- `write_extern();`
- `}`

support.cpp

- `#include <iostream>`
- `extern int count;`
- `void write_extern(void) {`
- `std::cout << "Count is " << count <<`
`std::endl;}`
- Here, *extern* keyword is being used to declare count in another file.

mutable Storage Class

- It allows a member of an object to override const member function. That is, a mutable member can be modified by a const member function.

A simple program

```
class A
{
    int val;

    public:
    A(int x)
    {
        val = x;
    }

    void changeVal(int x) const
    {
        val = x; //Error: cannot change val
    }
};
```

```
int main()
{
    const A a(10);
    a.changeVal(20);
}
```

Mutable Variables

```
class A
{
    mutable int val;

    public:
    A(int x)
    {
        val = x;
    }

    void changeVal(int x) const
    {
        val = x;
    }
};
```

```
int main()
{
    const A a(10);
    a.changeVal(20);
}
```

Same with static

```
class A
{
    static int val;

    public:
    A(int x)
    {
        val = x;
    }

    void changeVal(int x) const
    {
        val++;
    }
};

int A :: val=0;
int main()
{
    const A a(10);
    a.changeVal(20);
}
```